1501116 & 15011001 Computer Programming

Chapter 10: Class

Academic Year 2026/01

School of Applied Digital Technology, Mae Fah Luang University

Revised version by: Patcharaporn Panwong
Paweena Suebsombut
Sujitra Arwatchananukul

Original version by: Surapong Uttama

Lesson Plan

Week	Weekly Summary
Week 10 (5-9 Oct 20206)	**Chapter 7:** Iteration Statement: Loop Part 2
Week 11 (12-16 Oct 2026)	**Chapter 8:** Methods
Week 12 (19-23 Oct 2026)	**Chapter 9:** Array
Week 13 (26-30 Oct 2026)	**Chapter 10:** Class and Object
Week 14 (2-6 Nov 2026)	Group Project Implementation
Week 15 (9-13 Nov 2026)	**Group Project Presentation** ★★★
Week 16 (16-20 Nov 2026)	**Group Project Presentation** ★★★
Final Exam (Thursday 26 November 2026, 09:00 - 12:00)	

FIRST SEMESTER 2026

The Online Teaching Evaluation System

NOW OPEN . . .

You can complete the online teaching evaluation for all subjects from now until November 22, 2026

Reminders

The students who do not complete the online teaching evaluation before November 8, 2026.
"The REG system will be locked."

Scan to Evaluate

Contact Info

Division of Quality Assurance and Curriculum Development.
AS Building, 4th Floor
E-mail : qa.eval@mfu.ac.th
Tel. 053 916 407



Link:
https://evaloutcome.mfu.ac.th/login

Outline

- Review: Array
- Object Oriented Programming
- Class & Instance
- Constructor
- Array of class



Review: Array

Array Declaration and Creation

- **Combine declaration & creation command**

```
string[] students = new string[3];
```

*** Then, **students array** contains **3 elements**.*

students[0], students[1], students[2]

students

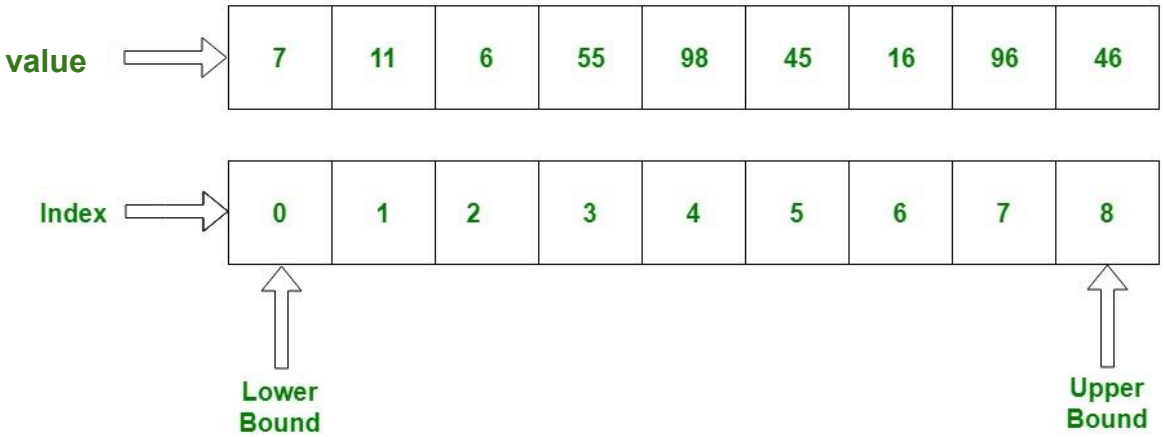
0	1	2

- *An array is a collection of values of the **same data type***
- *Individual variables are called elements*
- *Elements accessed through an index*
- *Index starts at 0*

Example: Initialization of Array

```
// defining array with 9 elements which indicates the size of an array
// int[] value  = {7, 11, 6, 55, 98, 45, 16, 96, 46};
```

```
int[] value  = new int[9] {7, 11, 6, 55, 98, 45, 16, 96, 46};
```



C# | Arrays - GeeksforGeeks

“foreach” and “for” loop in Array

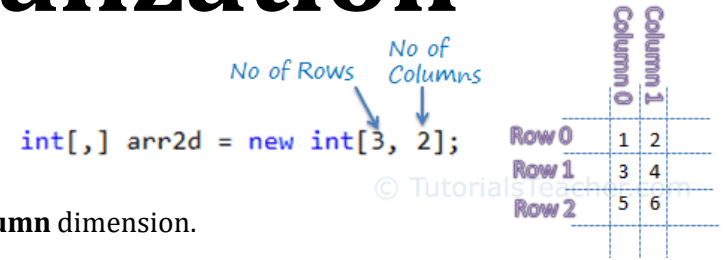
Normally, using “**foreach**” no need of using a counter

```
int[] numbers = { 4, 5, 6, 1, 2, 3, -2, -1, 0 };
foreach (int i in numbers)
{
    Console.Write("{0} ", i);
}
// Output: 4 5 6 1 2 3 -2 -1 0
```

Normally, using “**for**” with array needs a counter

```
int[] numbers = { 4, 5, 6, 1, 2, 3, -2, -1, 0 };
for(int i =0; i< numbers.Length; i++)
{
    Console.Write("{0} ", numbers[i]);
}
// Output: 4 5 6 1 2 3 -2 -1 0
```

2D array with initialization



- Single-dimensional array → 1D → 1 index → vector
- Two-dimensional array → 2D → 2 indexes → table, matrix
- In a 2D array, the **first** index is the **row** dimension, the **second** index is the **column** dimension.

Example:

- `int[,] a = { {11,12} , {21,22} };`

11	12
21	22

col 0 / col 1

row 0	a[0,0]	a[0,1]
row 1	a[1,0]	a[1,1]

- `int[,] a = { {11,12,13} , {21,22,23} };`

col 0 / col 1 / col 2

row 0	11	12	13
row 1	21	22	23

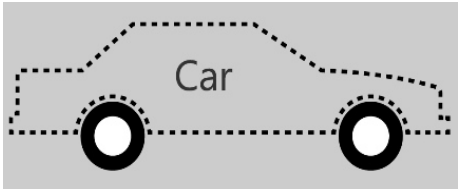
Object Oriented Programming

Object Oriented Programming(OOP)

What is Class?

- Are like blueprints for objects
- Classes model real-world objects and define
  - **Attributes** (state, fields)
  - **Behaviour** (methods, functions, operations)
- When use, an instance of class (object) is created
- A one class can be used to create several objects

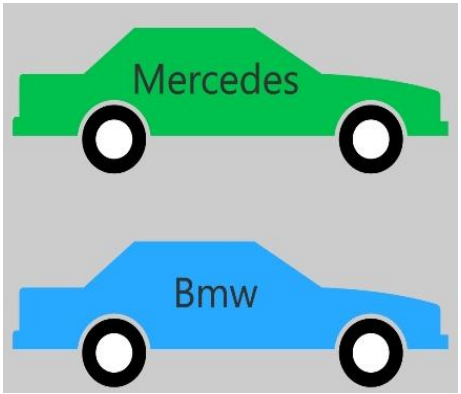
Class



What is Object?

- Are instances of a class
- Created using the **new** keyword
- Have actions

Object



Picture from PHP Object-Oriented (w3big.com)

Object oriented vs Procedural programming

Procedurl

- Procedural programming is a list or set of instructions telling a computer what to do step by step and how to perform from the first code to the second code.
- A systematic order of statements, functions and commands
- Not suitable for large scale system

```
class test {
            public static void Main() {

            Console.WriteLine("Hi");
            }
}
```

OOP

- Object-oriented programming models the real world in terms of objects. Everything in the world can be modeled as an object.
- In .NET Framework the object-oriented approach
- Classes can create several instances at the same time

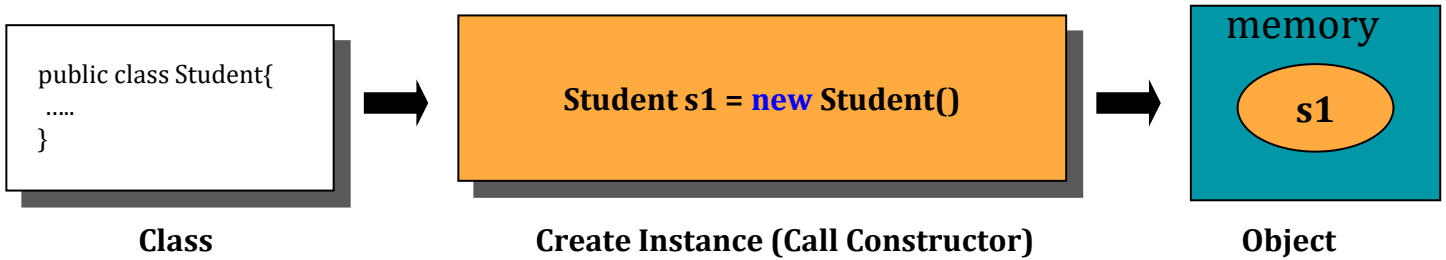
Class & Instance

Classes in C#

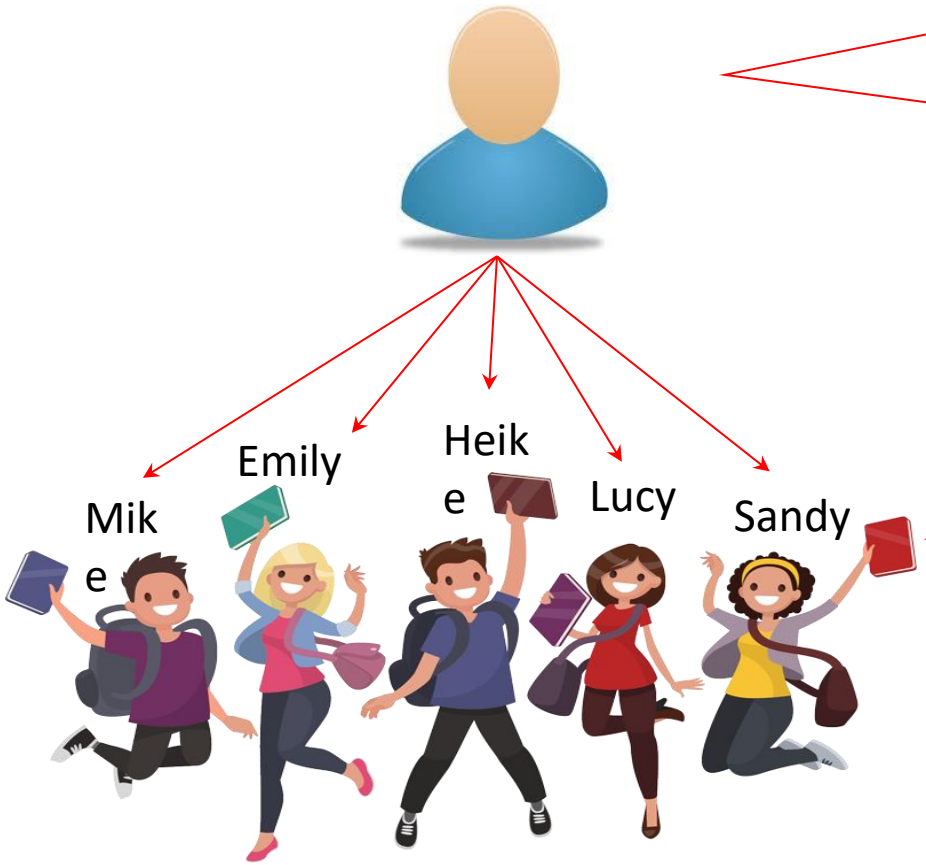
- Classes model real-world objects and define
  - Attributes (state, fields)
  - Behaviour (methods, function, operations)
- Classes describe the structure of objects
  - Objects describe particular instance of a class
- Properties hold information about the modeled object relevant to the problem
- Operations implement object behaviour

Class and Object

- Class identify the object
- Class is a blueprint of the objects
- Class is Abstract
- Object is Real



Example of Class & Objects



Class: Student

Attributes

-Name
-Gender
-Hair color
- Weight
- Height
- Personality

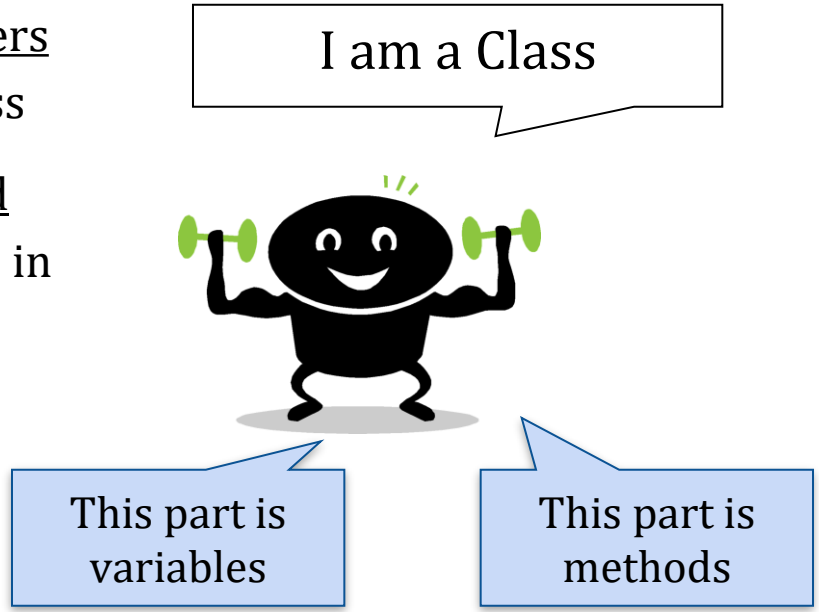
Behavior: Eat, Dance, Sleep

Object: Sandy

-**Name:** Sandy
-**Gender:** Female
-**Hair color:** Red
-**Weight:** 45
-**Height:** 160
-**Personality:** Nice

Encapsulation

- **Class contains**
  - **Field**(states, attributes) - data members which are **variables** declared in a class
  - **Method** (actions, behaviors) - method members which are **methods** defined in a class



Modifier

C# have information hiding by using modifier which are applied on both attribute & method

Modifier	Description
public	The code is accessible for all classes
private	The code is only accessible within the same class

access-modifiers:Microsoft.com

Field

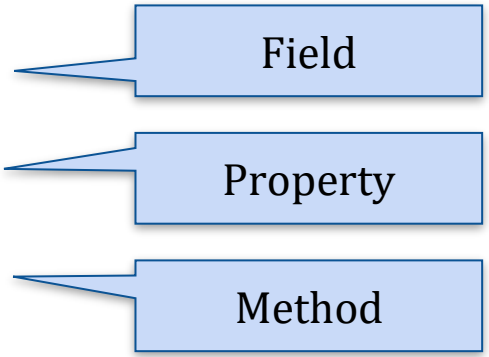
- Fields are variables declared at **Class Scope**.
  - Fields typically store the data that must be accessible to more than one type method and must be stored for longer than the lifetime of any single method.
- Generally, you should use fields only for **variables** that have **private** or **protected** accessibility.
- A **private field** that stores the data exposed by a **public property** is called **Backing field**

```
class Car {
        string color;
        int maxSpeed;
}
```

Class Declaration

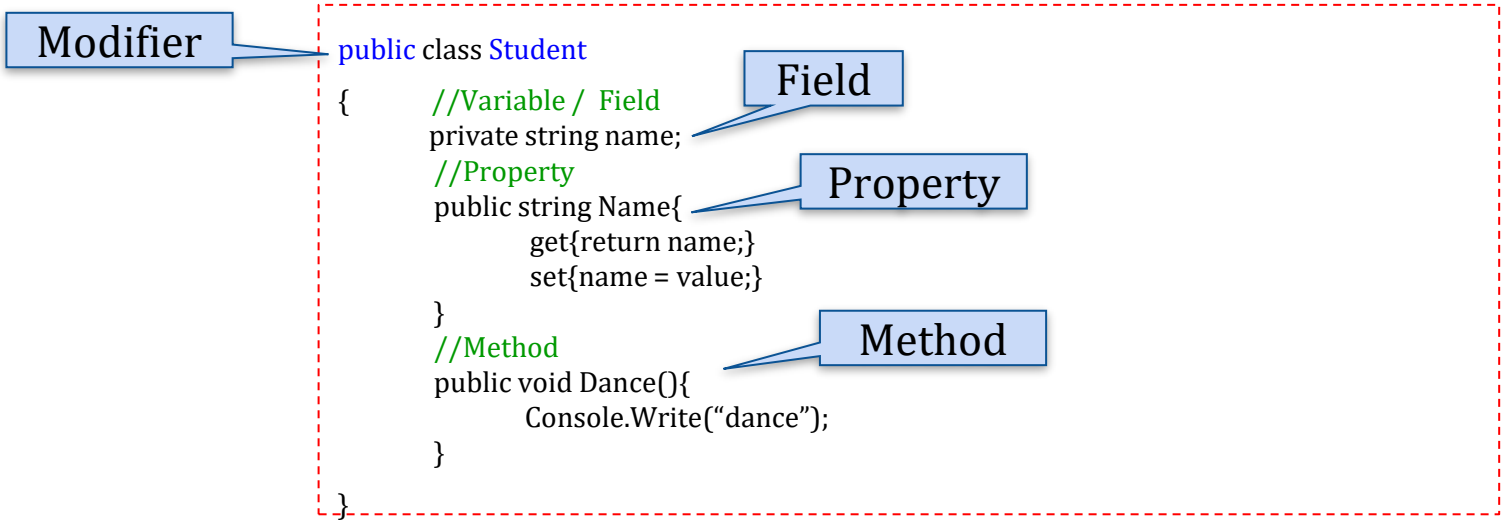
Class comprises a class header and class body.

```
modifier class class_name
{       //class body
        [variables declaration;]
        [property declaration;]
        [methods declaration;]
}
```



How to Define a Class and Create an Object

- **Define a class**



- **Create an object / an instance of a class**

```
Student toby = new Student();
```

- **Access class method**

```
toby.Dance();
```

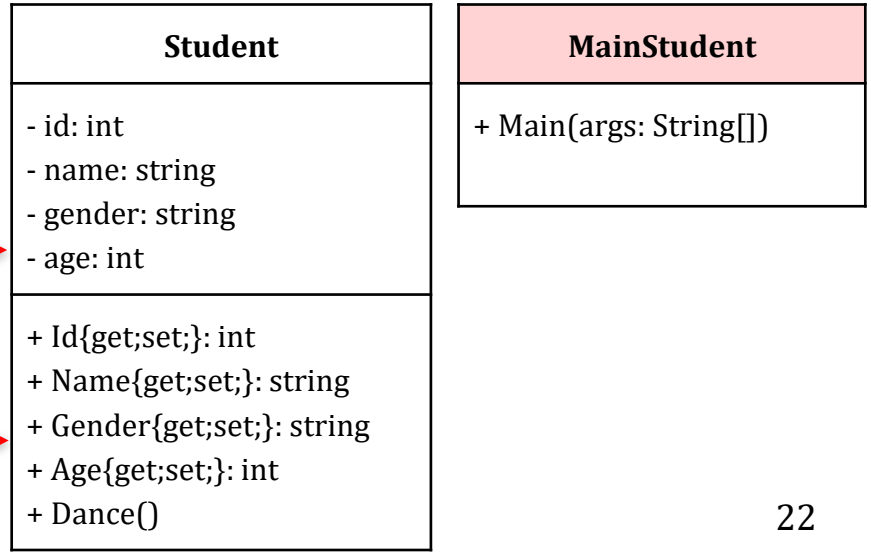
UML's Class Diagram

Class Diagram is created to represent the design to the programmer, it explain

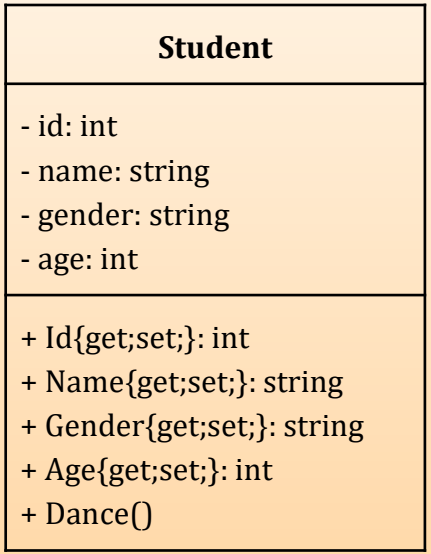
- What class do we have?
- How does each class look like? (attribute/method)
- How do these classes are related?

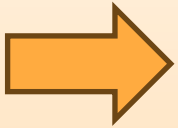
- \- means private
- \+ means public

- Attributes – data members
- Properties/Methods



Transform Class Diagram to Code





Example

```
using System;
public class Student
{
    private int id;
    Field: name
    }
}
```

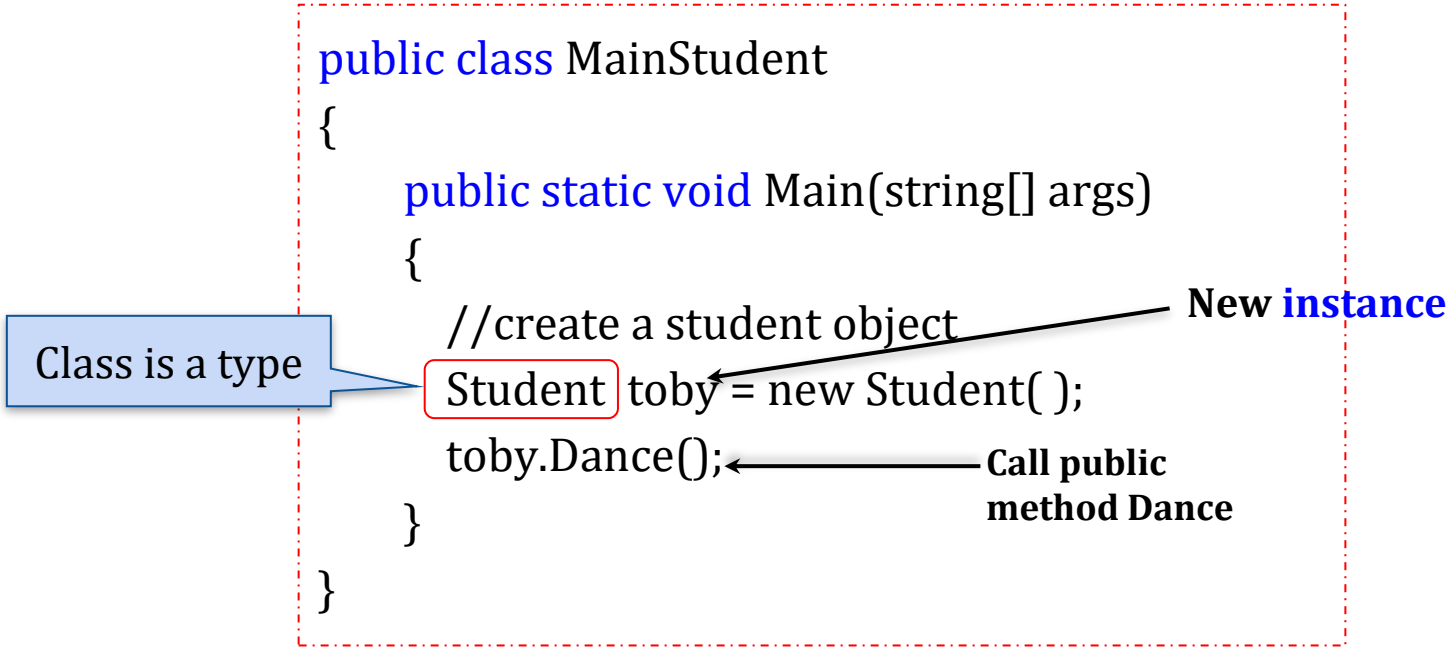
A. private string name;
B. private name string;

Multiple Choice

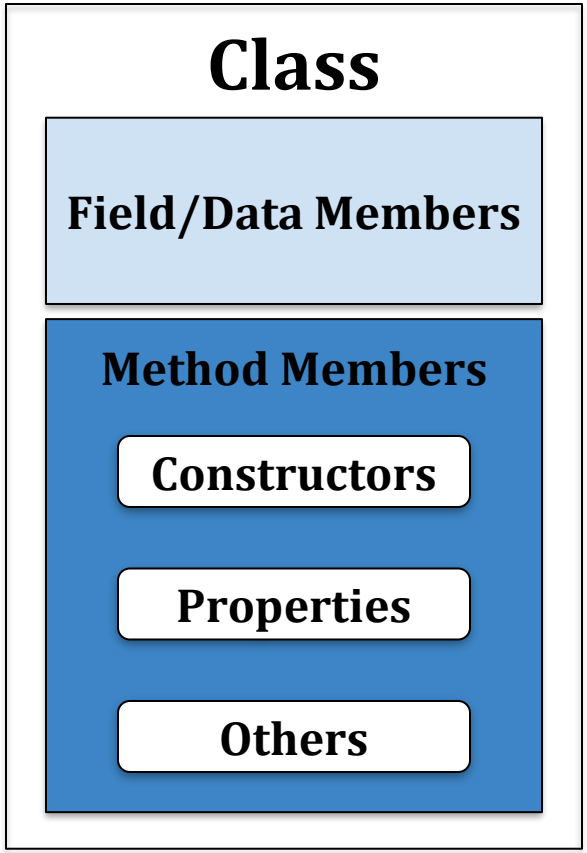
Using Student Class

When class is in used.

Class is a type, Instance (object of a class) is a variable.



Class Anatomy



- **Constructor** – initialize data members when new instance is created

- **Properties:**
  - **Setter** – write value to a data member
  - **Getter** – return value of a data member

Properties

- Like a combination of “a variable and a method”
- Use to access private fields
- Provide protection for a class field → to keep it from being changed without the knowledge of the object.
- It has two methods: a get accessor and a set accessor
  - **Get**: Get value from field
  - **Set**: Ser value to field
- The **“value”** keyword is used to define the value being assigned by the set

3 Ways for Writing Properties

1. **Custom**: Allow you to perform some conversion / computation on data before it is stored or returned.
2. **Auto implemented**: Just assign a value to OR retrieve a value from a backing field without including any additional logic
3. **Short-hand and Auto implemented**:
   - Field automatically created
   - Do not have to define the field for the property
   - Only have to write get; and set; inside the property.
   - Simplify your code

Sample C# Properties

```
class Car {
    private string color; //private field
    private string name; //private field
    public string Color{ //property : custom
        get {return color;}
        set {color=value;}
    }
    public string Name{ //property : auto-implemented
        get => name;
        set => name = value;
    }
    public int Age { get; set; } //short-hand / auto-implemented
}
```

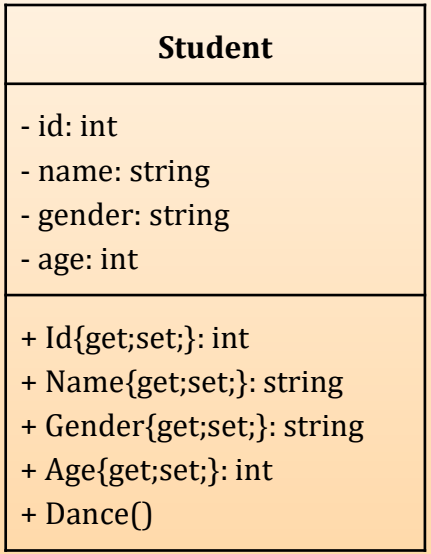
Private Backing Fields

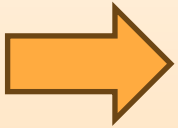
Custom

Auto implemented

Short-hand Auto implemented

Transform Class Diagram to Code





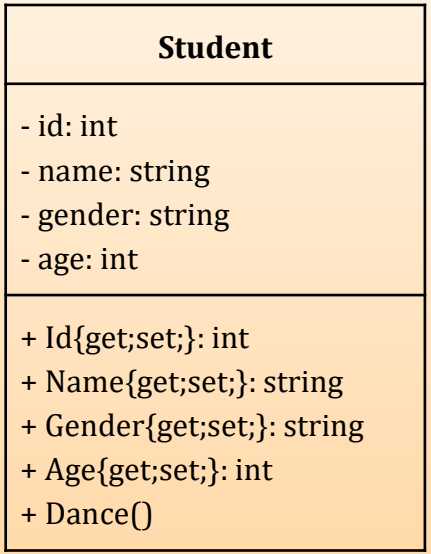
Example

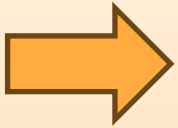
```
using System;
public class Student{
    ..... //field declaration
    public int Id {//property : custom
        get { return id; }
        set { id = value; }
    }
    public string [ ] {//property : custom
        get { return name; }
        set { [ ] }
    }
    .....
}
```

A. name
B. Name

Multiple Choice

Transform Class Diagram to Code





Example

```
using System;
public class Student{
    ..... //field declaration
    public int Id {//property : custom
        get { return id; }
        set { id = value; }
    }
    public string Name {//property : custom
        get { return name; }
        set {          }
    }
    .....
}
```

Multiple Choice

A. value = Name;
B. Name = value;
C. name = value;
D. value = name;

Exercise #1

```
public class MainCar
{
      public static void Main(string[] args)
      {
            Car car1 = new Car();
            car1.Color = "Blue";
            car1.Name = "Honda";
            car1.Age = 5;

            Console.WriteLine("Car color = {0}, brand = {1}, age = {2} years“
            , car1.Color, car1.Name, car1.Age);
       }
}
```

```
class Car
{
   private string color; //private field
   private string name; //private field
   public string Color
   { //property : custom
     get { return color; }
     set { color = value; }
   }

   public string Name
   { //property : auto-implemented
     get => name;
     set => name = value;
   }

   //short-hand and auto-implemented
   public int Age { get; set; }
}
```

What is the result?

Result

Constructor

Constructor

- Constructor: a class member(method) with the same name as class name, no static, and no return data
- **Constructor is used to**
  - Initialize an instance of a class by setting default or initial values for its data members immediately after the object is created.
- One class can contain multiple constructors which get different arguments.
- When the constructor *is not* declared, the default is a constructor with no parameters.

Constructor (Continue)

Methods with

- Same name as class name
- No return type defined

```
public class Person
{
    private string last;
    private string first;

    public Person()
    {
      last = null;
      first =  null;
    }
    public Person(string lastName)
    {
      last = lastName;
    }
    public Person(string lastName, string firstName)
    {
      last = lastName;
      first = firstName;
    }
}
```

Multiple Constructors with different set of parameters

Using Constructor

```
public class Person
{
    private string last;
    private string first;

    1 reference
    public Person()
    {
        last = null;
        first = null;
    }
    1 reference
    public Person(string lastName)
    {
        last = lastName;
    }
    1 reference
    public Person(string lastName, string firstName)
    {
        last = lastName;
        first = firstName;
    }
```

```
public class MainPerson
{
    0 references
    public static void Main()
    {
        //create object of Person
        Person p1 = new Person();

        //create Toby of Person
        Person p2 = new Person("Toby");

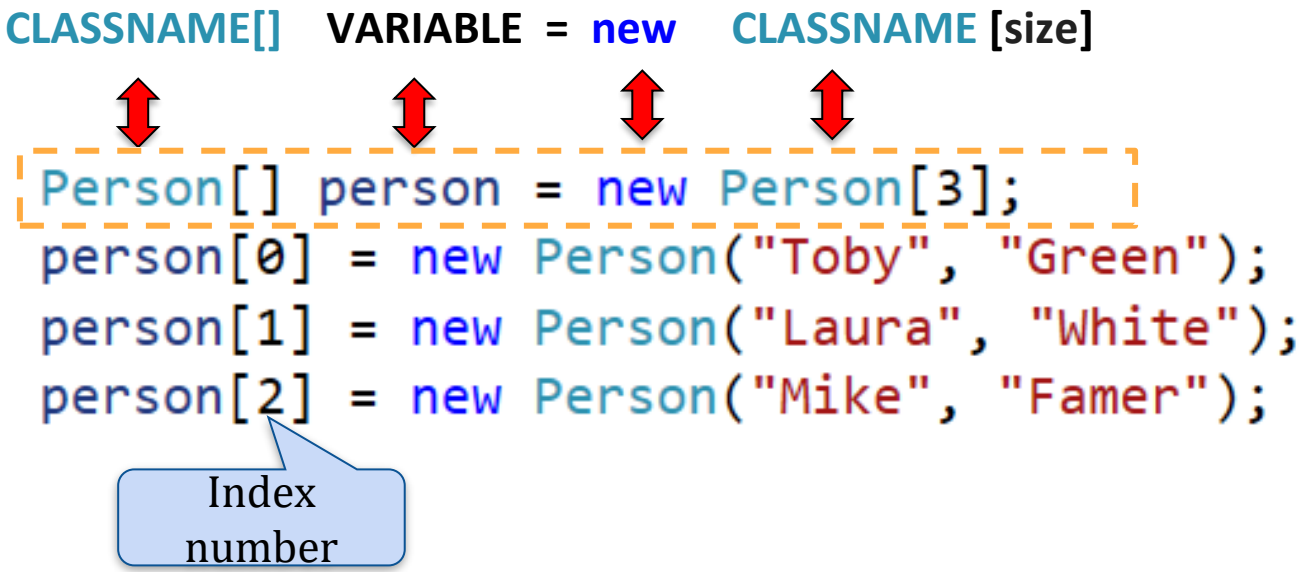
        //create Toby of Person
        Person p3 = new Person("Laura","White");


    }
}
```

Array of Class

Array of Class

- Class is just like the other type
- Array can be declared with any class too



Summary

- What is Class?
- What is Object?
- What modifier do we have?
- What is Constructor?

Announcement

Group Project (25%)

- Num# students **1 group = 3 - 5 students**
- Form a group with only your classmates from same lecture class
  - Example: Your lecture is on Monday (ONLY section 1)

- Present Project: **10 minutes /group**
- Question and Answer: **5 minutes /group**

- **Console application (No GUI interface)**
  - **Any topics** e.g., noodle shop, library, ticket reservation, and so on

The Group Project Evaluation Matrix (25% or 25 Points)

Computer Programming Subject: Project Rubric (25%)

if showing up on presentation day but does not present --> presentation score = 0
if missing for presentation day --> presentation score = 0
if missing and/or doing nothing for project (reported by your teammate) --> group project = 0

Evaluation Criteria									Total (25 pts)
Completeness **(5 pts)**	Logic & Technical Basic						**Complexity (2 pts)**	Presentation **(3 pts)**	
	if-else **(2 pts)**	switch case **(2 pts)**	Function(s) / Method(s) **(4 pts)**	Loop **(3 pts)**	Array **(2 pts)**	Add input data to array **(2 pts)**			
[1]. all complete(no error) = 5, [2]. logic error = 3, [3]. project cannot execute/run = 1	-1 when there is no statement inside	-1 when there is no statement inside	-1 when there is no calling from Main()	-1 when repeat only one round	-1 when contains only one element	-2 if no data is entered for each array element	Basic (similar to the lab) = 1 points Extended from the lab with full functions (Self-study / Wow) = 2 points	-1 if can't answer all questions -1 if begin your presentation late or go over time Remark: Case1: missing presentation = 0 points Case2: showing up on presentation day but does not present = 0 points	

To understand the grading fully, please read the Group Project Rubric here.

Final Project Presentation (25%)

- To understand the grading fully, please read the Group Project Rubric here.
  - Remark: Do not forget to read FAQ for group project & presentation!!!

Condition	Score	Outcome
Project Missing (No group and No presentation)	-25	The student does not belong to any group, does not attend and present on presentation day, and has no project submission. **The Group Project Score = 0.**
Project Missing + Non-Participation (No contribution)	-25	The student belongs to a group, but the student is reported by a teammate as having done nothing for the project. **The Group Project Score = 0.**
Presentation Absence	-3	If the student is missing for presentation day but other members confirm that you helped and worked on the project, **your Presentation Score = 0.**
The student who shows up on the presentation day but does not present.	-3	Every student in the team must present. **Any student who does not present will receive a Presentation Score of 0.**



Q & A

